

Big Data Analytics

Unit 1 Part 1



Introduction to Big Data

Introduction to Big Data

- Big data is an all-inclusive term that refers to extremely large, very fast, highly diverse and complex data that cannot be managed with traditional data management tools
- It includes all kind of data, which helps deliver the right information, to right person, in right quantity, at right time, to make right decision
- Big data can be harnessed by developing infinitely scalable, flexible, and evolutionary data architectures, coupled with the use of cost-effective computing machines.

Definition

“Big data” is

*high-volume, -velocity and -variety information
assets*

*that demand cost-effective, innovative forms of
information processing*

for enhanced insight and decision making

By Gartner

Understanding Big Data

- Big data is data that exceeds the processing capacity of conventional database systems.
- The data is too big, moves too fast, or doesn't fit the structures of your database architectures.
- To gain value from this data, you must choose an alternative way to process it.
- At the fundamental level it is just another collection of data that can be analyzed and utilized for the benefit of the business and on another level, it is a special kind of data that poses unique challenges and offers unique benefits.

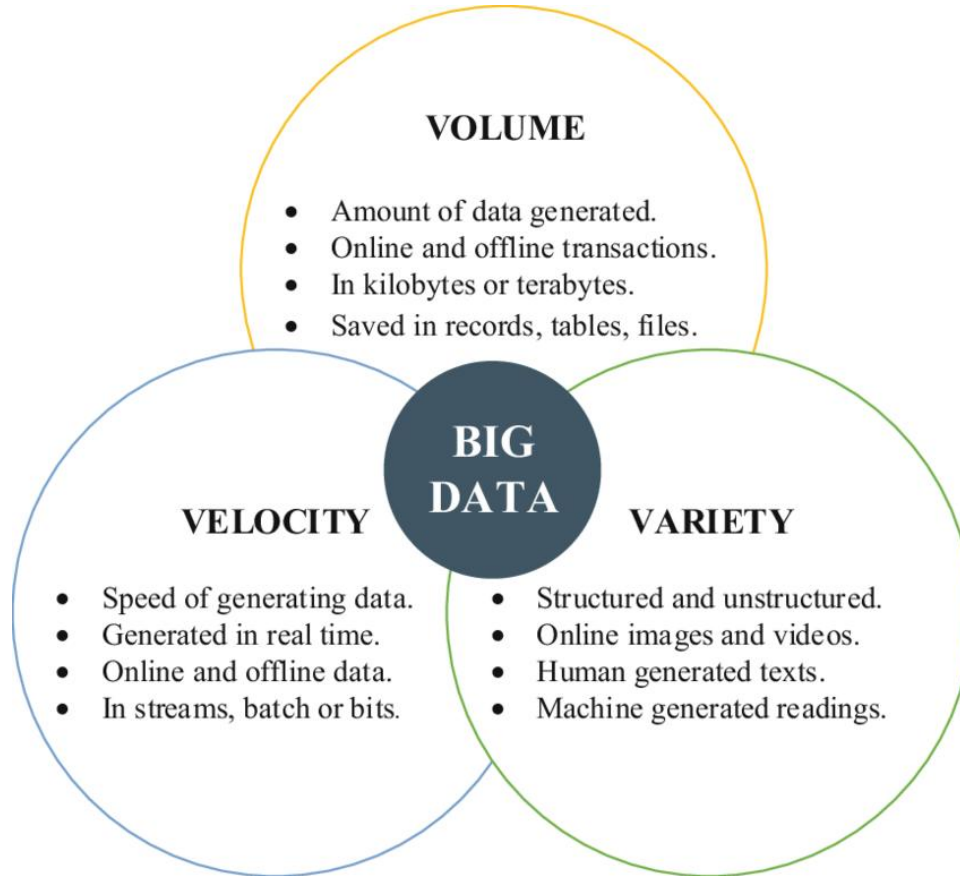
Life Cycle of Big Data

Big data is mostly, over 90%, unstructured data

There are huge opportunities for technology providers to innovate and manage the entire life cycle of Big Data

- Generate
- Gather
- Store
- Organize
- Analyze
- Visualize

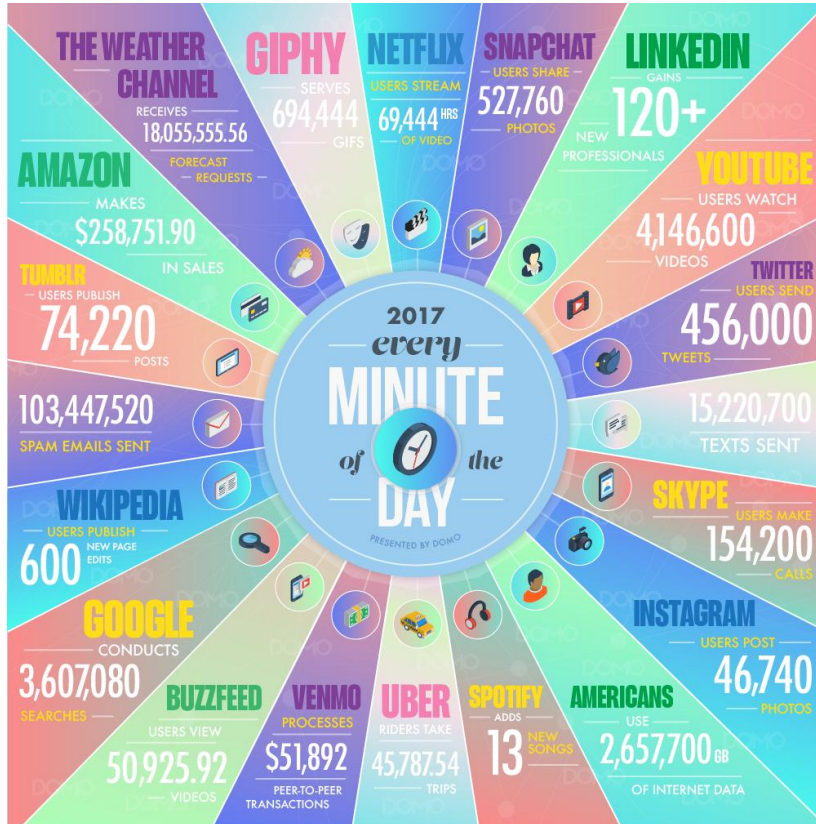
The Three V-s



Examples of Data Volumes

Unit	Value	Example
Kilobytes (KB)	1,000 bytes	a paragraph of a text document
Megabytes (MB)	1,000 Kilobytes	a small novel
Gigabytes (GB)	1,000 Megabytes	30 Minutes of HD Video Streaming
Terabytes (TB)	1,000 Gigabytes	all the X-rays in a large hospital
Petabytes (PB)	1,000 Terabytes	half the contents of all US academic research libraries
Exabytes (EB)	1,000 Petabytes	about one fifth of the words people have ever spoken
Zettabytes (ZB)	1,000 Exabytes	as much information as there are grains of sand on all the world's beaches
Yottabytes (YB)	1,000 Zettabytes	as much information as there are atoms in 7,000 human bodies

Volume

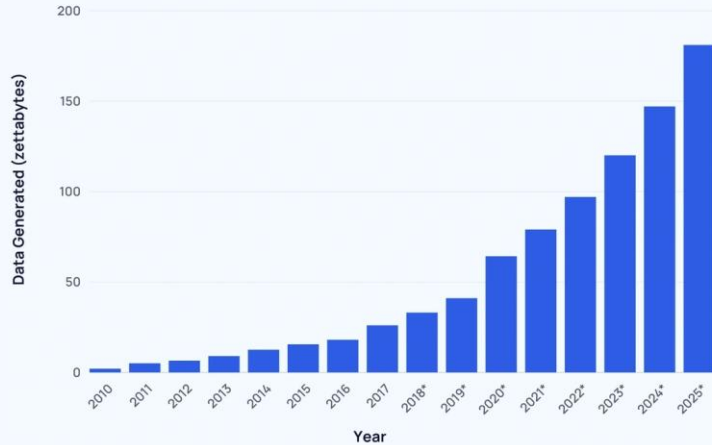


- Approximately **402.74 million terabytes** of data are created each day
- Around **221 zettabytes** of data is expected to be generated in 2026
- **181 zettabytes** of data were generated in 2025
- Videos account for **~82%** of internet data traffic

Image Source: <https://www.domo.com/blog/data-never-sleeps-5/>
<https://explodingtopics.com/blog/data-generated-per-day>

Volume

Global Data Generated Annually



2025*

181 zettabytes

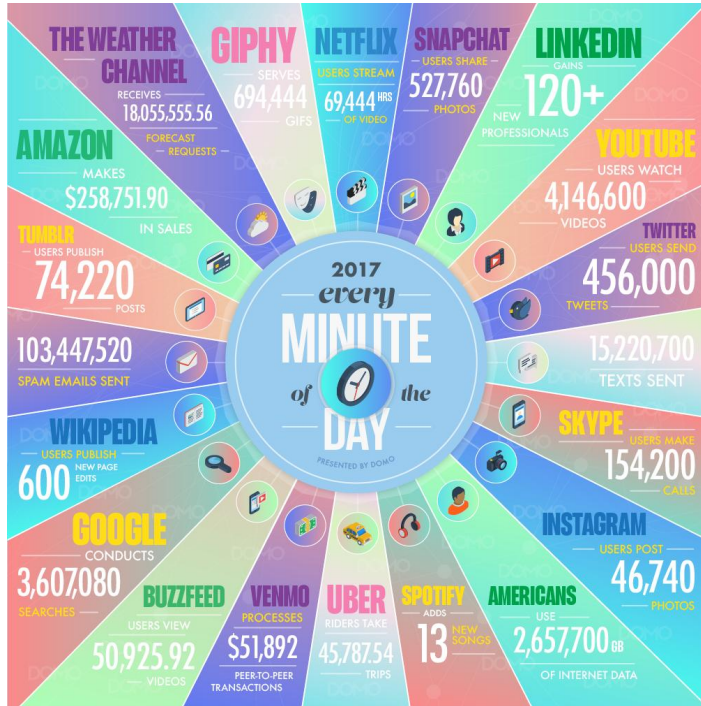
- Data generation is currently skyrocketing.
- And with every passing year, we are seeing these figures reach new highs.

- **Daily Output:** ~400–500 million terabytes (~0.5 zettabytes) created globally every day.
- **Rapid Expansion:** Exponential growth driven by AI, cloud computing, and IoT devices.
- **Social Media:** 100+ million daily Instagram uploads; 65+ billion WhatsApp messages.
- **Video Content:** 4+ million hours of new video uploaded to YouTube every 24 hours.
- **Global Messaging:** 294+ billion emails sent and received daily across the world.
- Approximately 90% of global data was created in just the last two years.

It only takes a minute...

- ✓ 16 million texts sent
- ✓ 5.9 million Google searches
- ✓ 231.4 million emails sent
- ✓ \$12,900 in online event bookings
- ✓ \$90.2 million in crypto purchases
- ✓ \$443, 000 spent on Amazon
- ✓ 1.1 million hours of streaming viewed
- ✓ 500 hours of YouTube videos uploaded
- ✓ 184,600 hours spent on Zoom meetings
- ✓ 1.7 million pieces of Facebook content shared
- ✓ 66,000 Instagram photos shared
- ✓ 347,200 tweets on Twitter
- ✓ 2.43 million snaps on Snapchat
- ✓ 1.1 million Tinder swipes

Velocity



- If traditional data is like a drop of water, Big Data is like flowing river
- Speed at which data is generated by billions of devices, and communicated at the speed of light. (High speed Internet availability)
- Mobile devices can generate and communicate data from anywhere, at any time
- Processing should be faster than generation
- Analyse data while it is being generated without even putting it into databases

Velocity (Speed)

- Data is being generated fast from different sources and need to be processed fast
- Online Data Analytics
- Late decisions → missing opportunities



- **Examples**

- **E-Promotions:** Based on your current location, your purchase history, what you like → send promotions right now for store next to you
- **Healthcare monitoring:** sensors monitoring your activities and body → any abnormal measurements require immediate reaction

Sources of Real-time/Fast Data



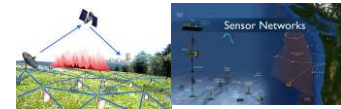
Social media and networks
(all of us are generating data)



Scientific instruments
(collecting all sorts of data)



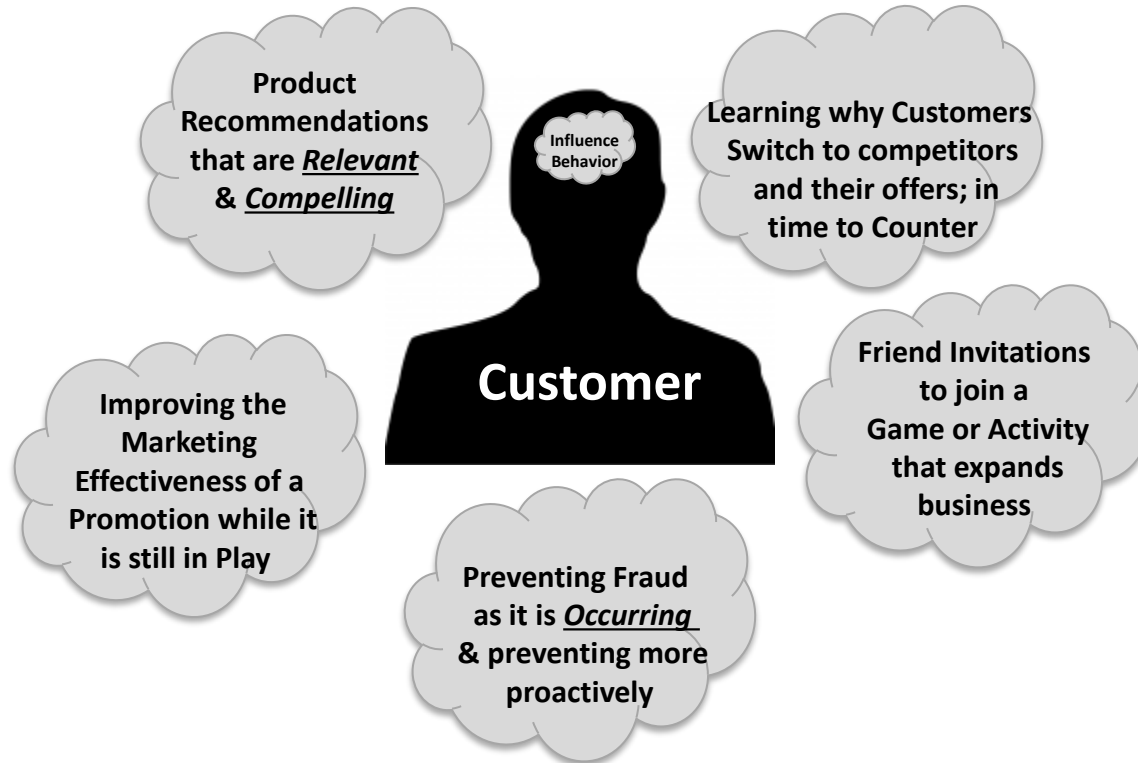
Mobile devices
(tracking all objects all the time)



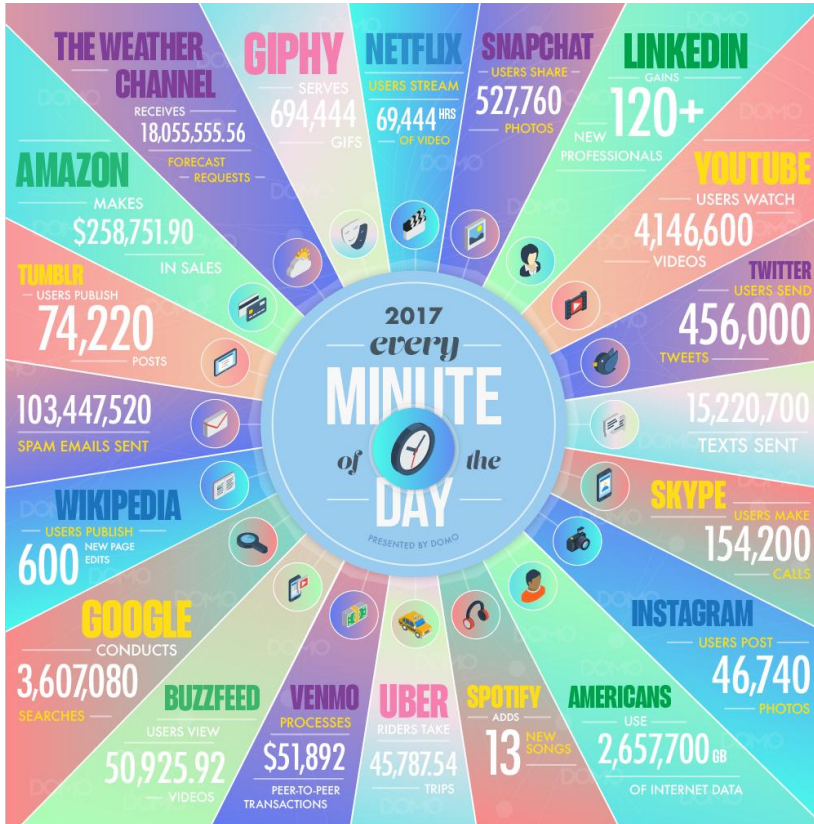
Sensor technology and networks
(measuring all kinds of data)

- The progress and innovation is no longer hindered by the ability to collect data
- But, by the ability to manage, analyze, summarize, visualize, and discover knowledge from the collected data in a timely manner and in a scalable fashion needs to be handled

Real-Time Analytics/Decision Requirement



Variety



- Different types of data that we can use is inclusive of all forms of data, for all kinds of functions, from all sources and devices
- Generated by different entities
 - Humans
 - Machines (HW + SW)
 - Sensors

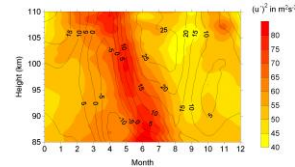
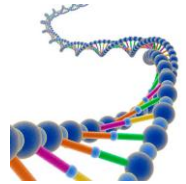
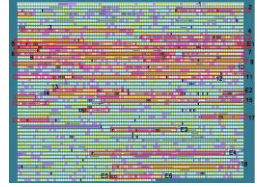
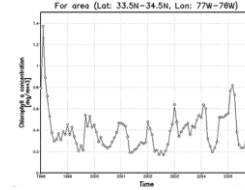
Image Source: <https://www.domo.com/blog/data-never-sleeps-5/>

Variety (Complexity)

- Relational Data (Tables/Transaction/Legacy Data)
- Text Data (Web)
- Semi-structured Data (XML)
- Graph Data
 - Social Network, Semantic Web (RDF), ...
- Streaming Data
 - You can only scan the data once
- A single application can be generating/collecting many types of data:
- Form Data type range in numbers to text, graph, map, audio, video and others
- Function: human conversation, songs and movies, business transaction records, machine operation performance data, new product design data, old archived data, etc.
- Source of data: Mobile phone and tablets, web access and search logs , Business transactional information, temperature and pressure sensors on machines, RFID tags on assets generate incessant and repetitive data.
- Big Public Data (online, weather, finance, detecting human faces from pictures, compare voice to identify the speaker, comparing handwritings to identify the writer, etc.)

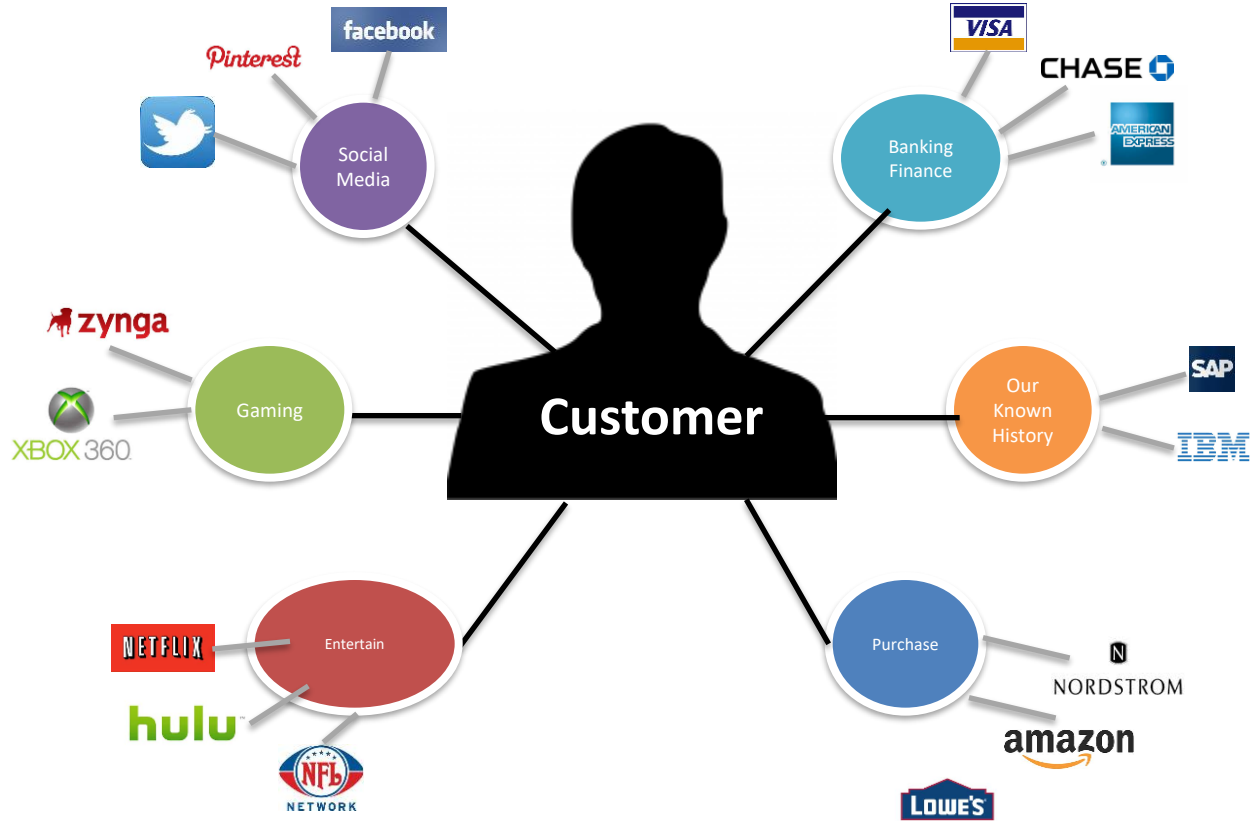
Broadly speaking there are three broad types of sources of data

- Human to Human Communication, Human-Machine communication and Machine to Machine communication.



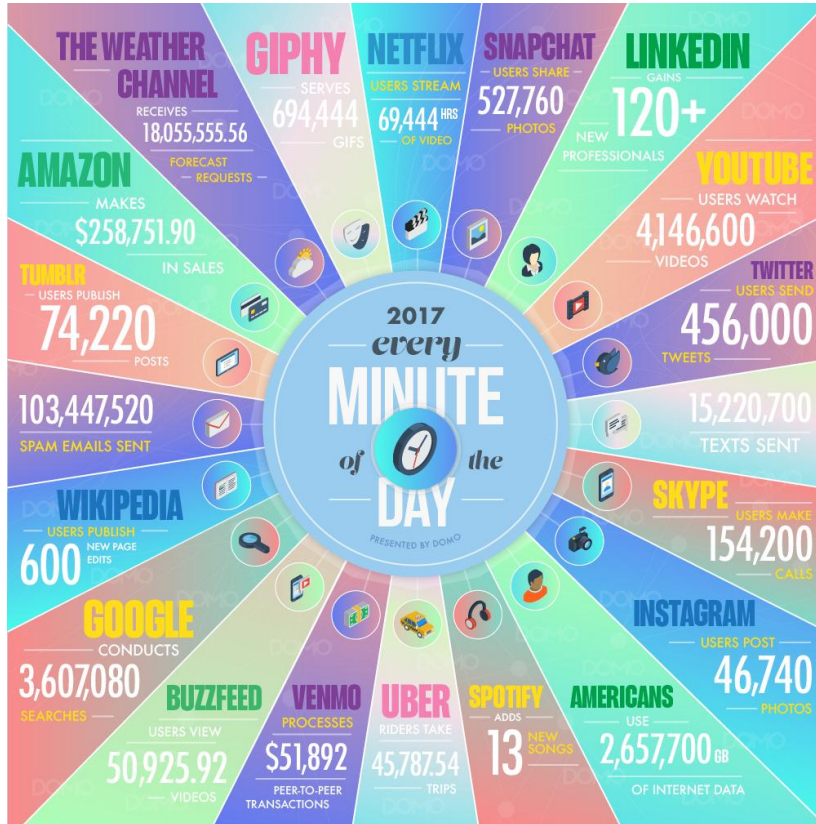
To extract knowledge → all these types of data need to be linked together

A Single View to the Customer



Additional V-s

Veracity



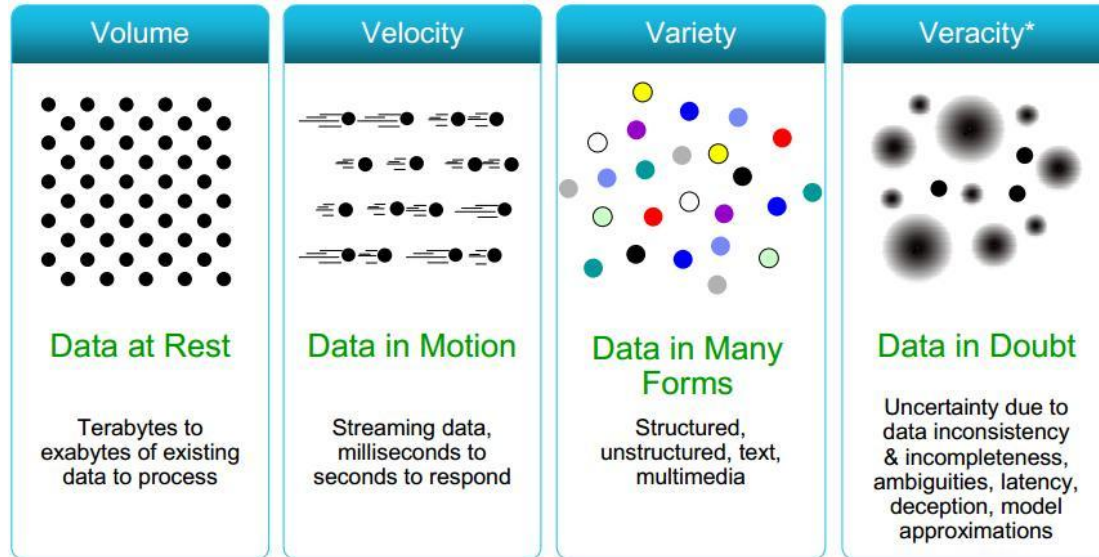
- Relates to the truthfulness, believability and quality of data
- Big Data is messy and there is considerable misinformation out there.
- The reasons for poor quality of data can range from technical error to human error to malicious intent.

Image Source: <https://www.domo.com/blog/data-never-sleeps-5/>

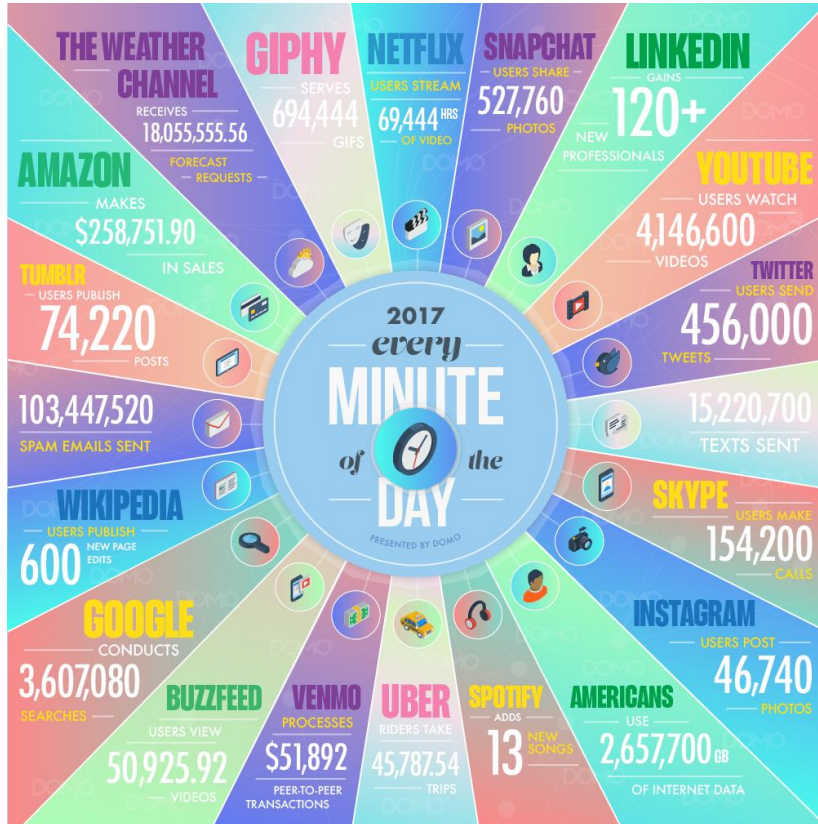
Veracity

- The source of information may not be authoritative
 - Information from government website (.gov) is authentic and complete, but Wikipedia data may not be equally reliable.
- The data may not be communicated and received correctly because of human or technical failure
 - Sensors and communication machines may malfunction and may record and transmit incorrect data.
 - Urgency may require the transmission of the best data available at a point in time. Such data makes reconciliation with later, accurate, records more problematic.
- The data provided and received may, however also be intentionally wrong for competitive or security reasons.
 - There could be disinformation and malicious information spread for strategic reasons.

Some Make it 4V's



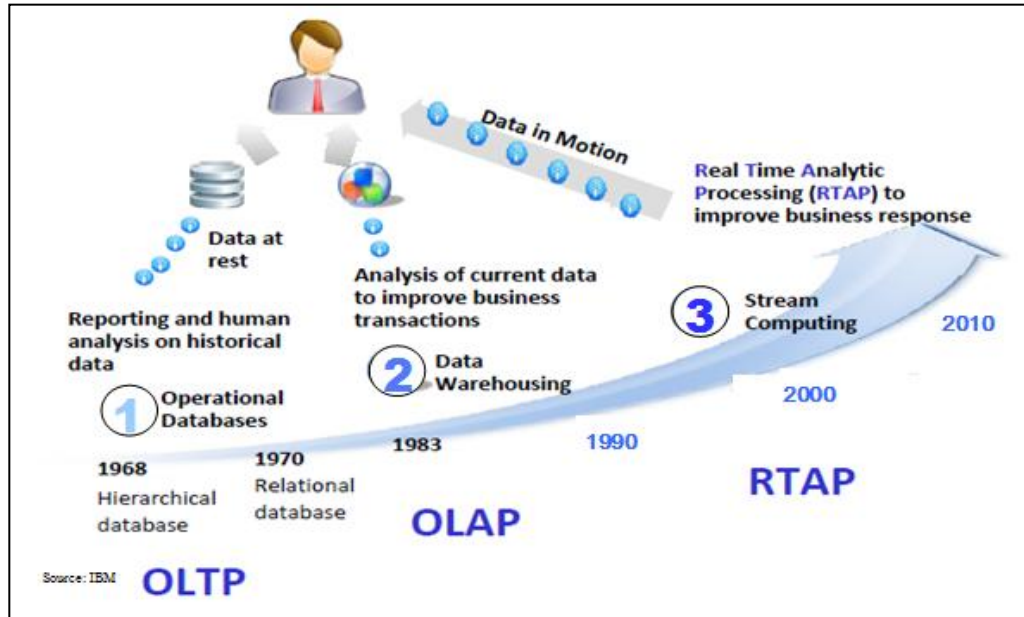
Value



Getting value out of Big Data!!!

Image Source: <https://www.domo.com/blog/data-never-sleeps-5/>

Harnessing Big Data



- **OLTP:** Online Transaction Processing (DBMSs)
- **OLAP:** Online Analytical Processing (Data Warehousing)
- **RTAP:** Real-Time Analytics Processing (Big Data Architecture & technology)

The Model Has Changed...

- **The Model of Generating/Consuming Data has Changed**

Old Model: Few companies are generating data, all others are consuming data



New Model: all of us are generating data, and all of us are consuming data



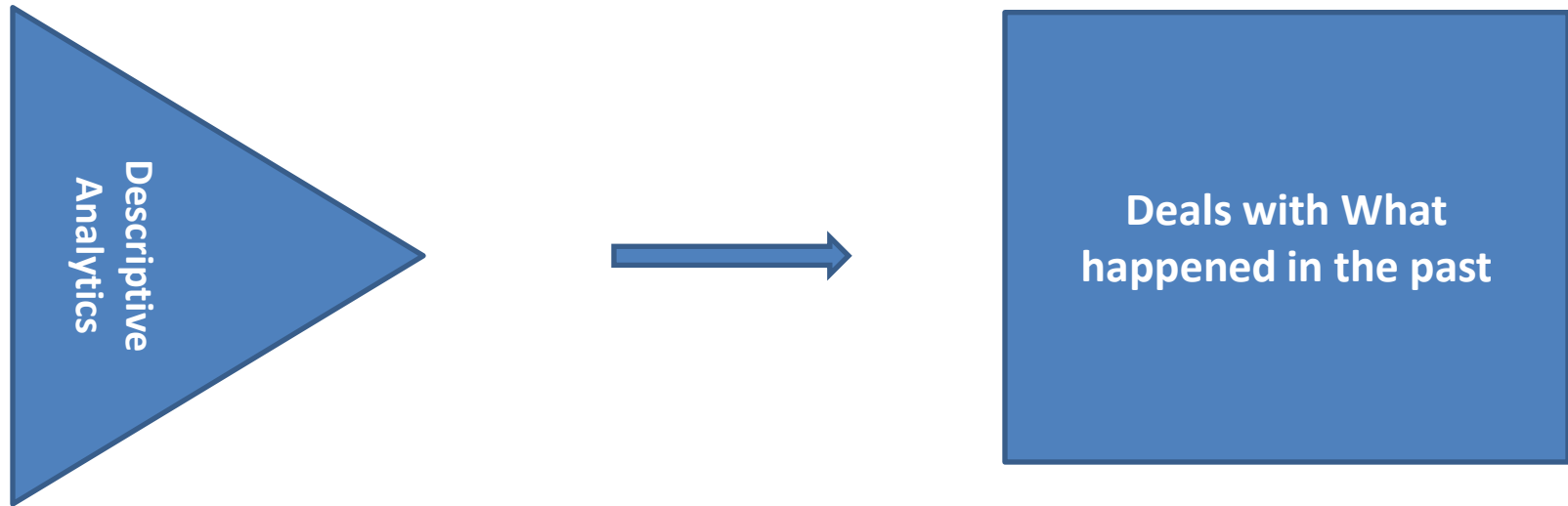
Types of Data Analytics

There are four major types of data analytics:

- 1. Predictive (forecasting)**
- 2. Descriptive (business intelligence and data mining)**
- 3. Prescriptive (optimization and simulation)**
- 4. Diagnostic analytics**

Descriptive Analytics: What happened?

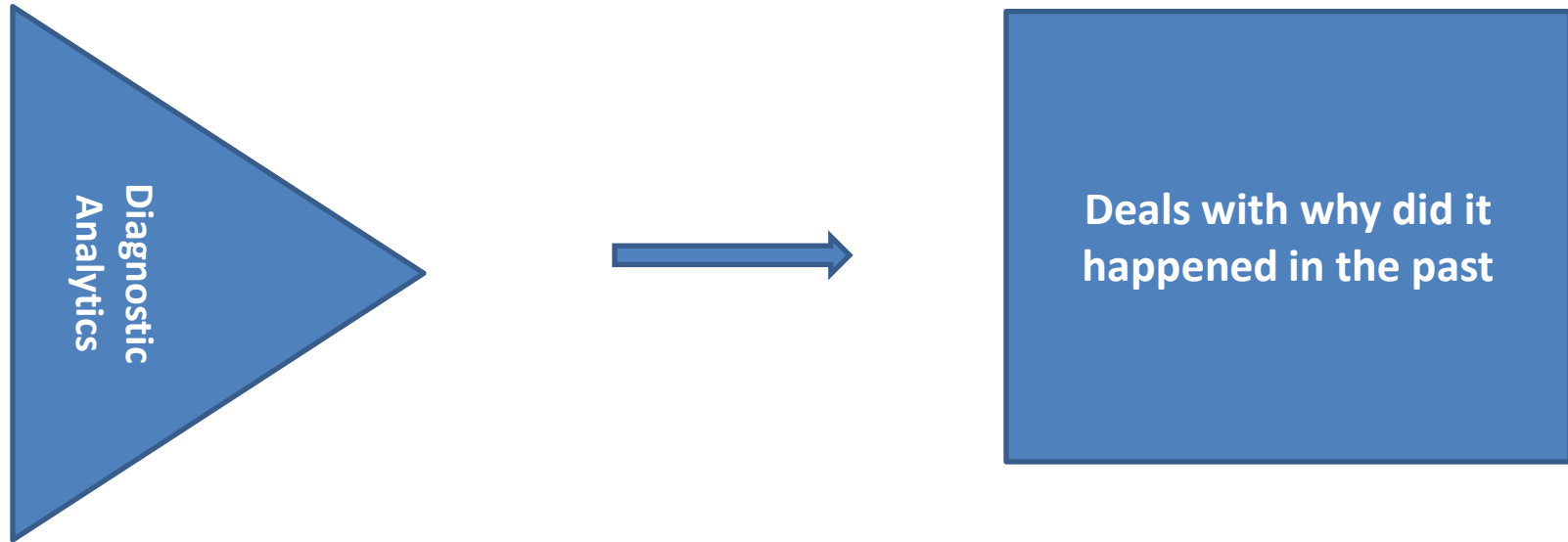
Descriptive analytics summarizes historical data using reporting, scorecards, and metrics to track past performance.



E.g. identifies many different relationships between customer and product.

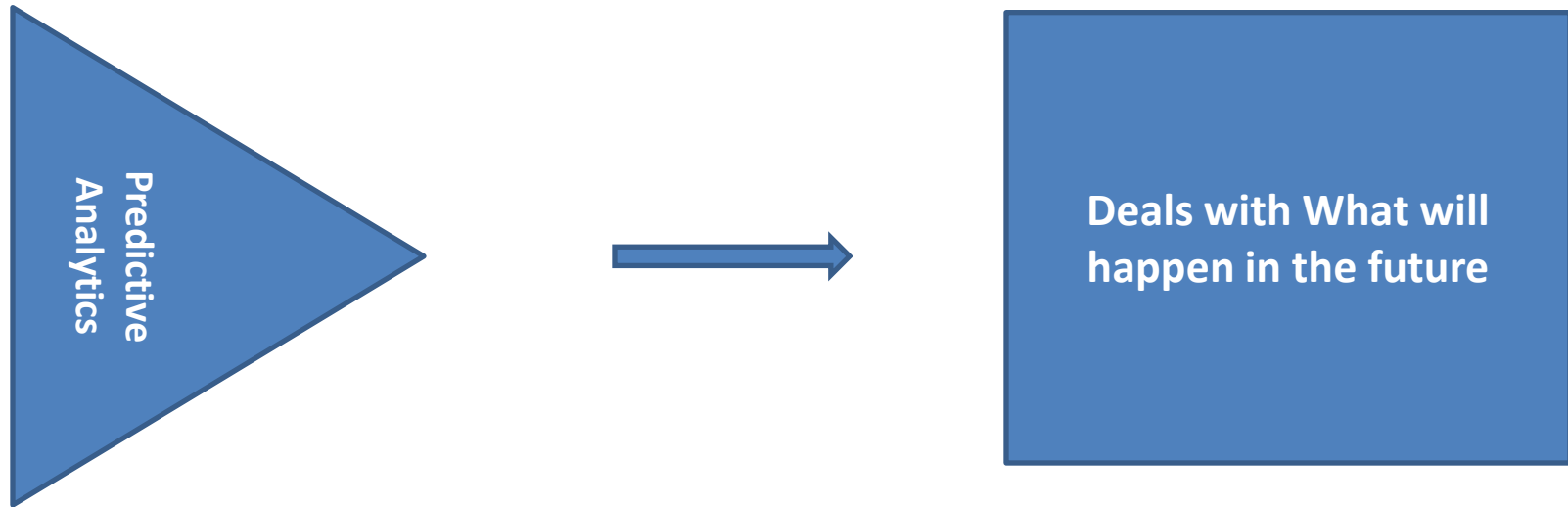
Diagnostic Analytics: Why did it happen?

Core focus: Investigates past data deeper to discover root causes, anomalies, or correlations behind specific trends.



Predictive Analytics (Forecasting): What is likely to happen?

Core Focus: Looks forward to anticipate future trends, behaviors, or risks based on historical patterns.

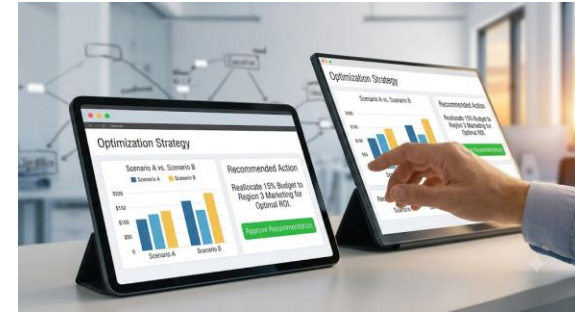
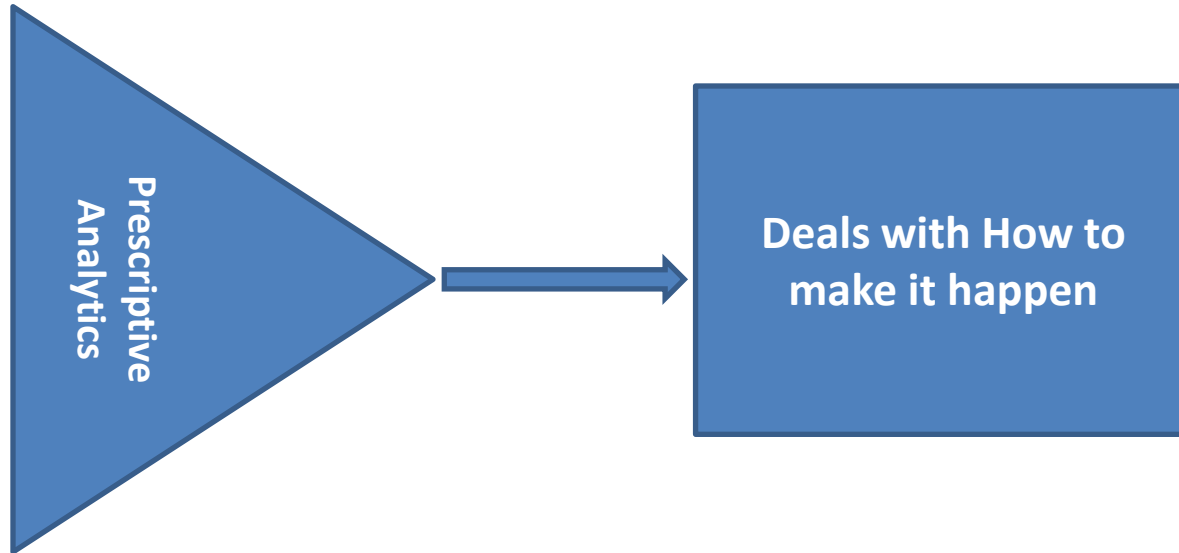


Predictive analytics turn the data into valuable, actionable information

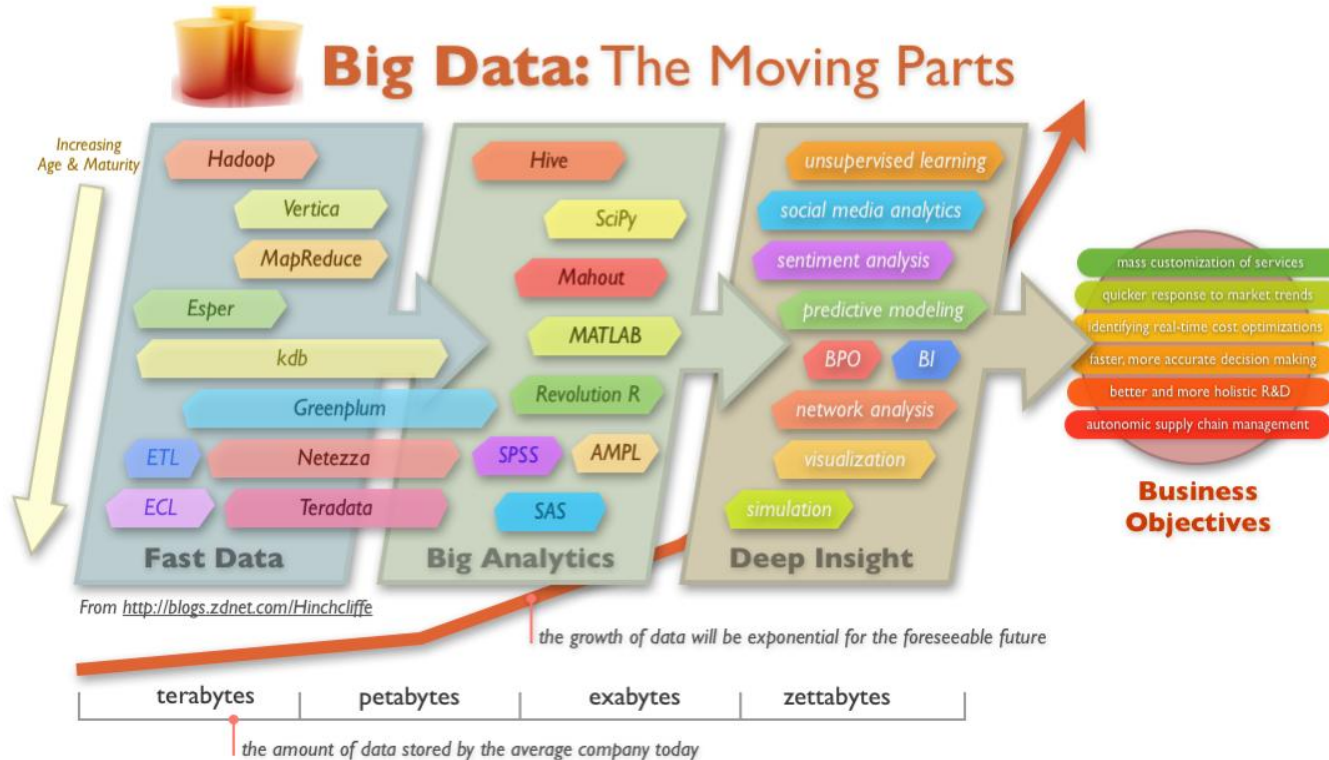
Prescriptive Analytics (Optimization & Simulation) :

What action should we take?

Prescriptive Analytics can suggest decision options on how to take advantage of a future opportunity or mitigate a future risk and illustrate the implication of each decision option.



Big Data Technology



Big Data Technologies

When dealing with Big Data, organizations generally process and store it in two distinct ways based on their business goals:

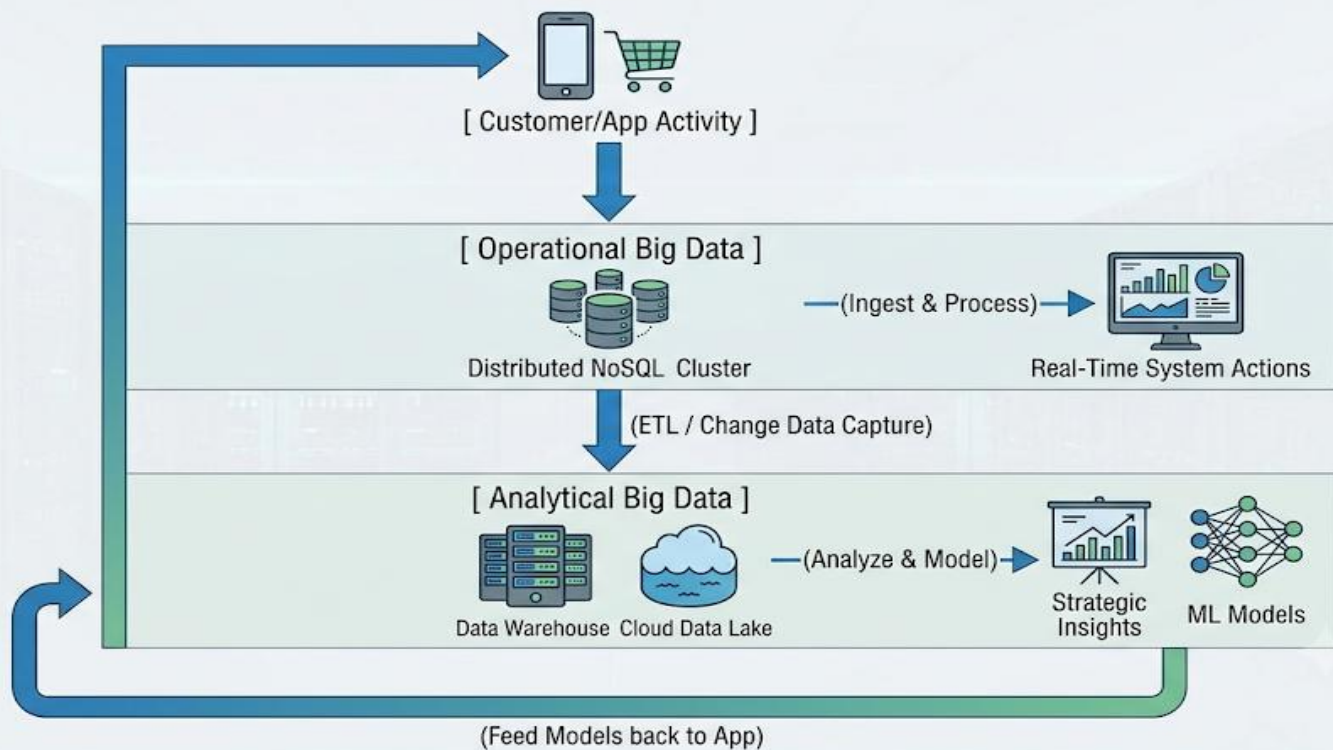
1. Operational Big Data (day-to-day operations) and
2. Analytical Big Data (long-term strategic insights)

Operational vs. Analytical Systems

Feature	Operational Big Data	Analytical Big Data
Primary Question	"What is happening right now?"	"What happened, why, and what will happen?"
Core Focus	Real-time processing & low-latency execution	Long-term trends, patterns, & strategic insights
Data Flow	Continuous, real-time transaction processing	Batch, stream, or micro-batch analytical processing
Database Type	NoSQL, Distributed OLTP	Data Warehouses, Data Lakes, OLAP
Typical Users	Front-end apps, customers, operational staff	Data scientists, business analysts, executives
Latency	Milliseconds to seconds	Seconds, minutes, or hours (depends on query size)

How They Work Together

Operational and Analytical Big Data are not mutually exclusive. They form a continuous feedback loop in modern data architecture:



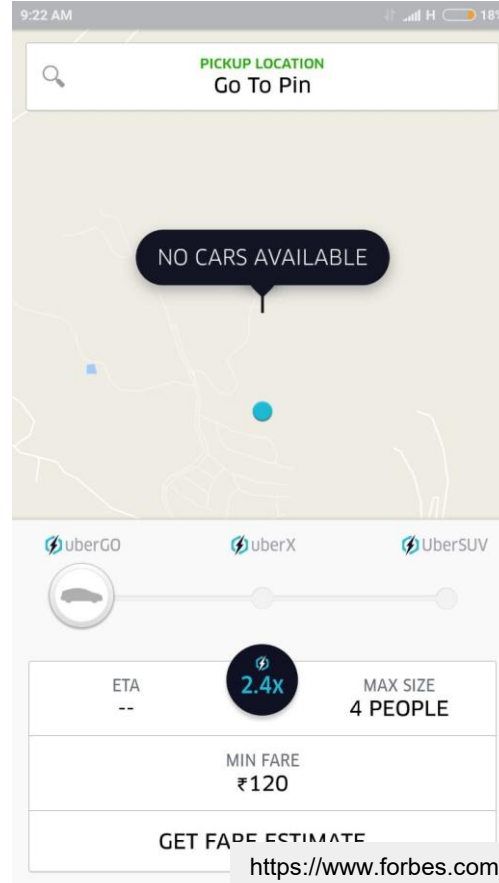
Use cases

Use Case: Big Data in Oil & Gas Drilling

- By applying big data analytics to sensor feeds monitoring offshore wells, the company replaced a costly \$5M, month-long physical inspection process with rapid data analysis.
- Data scientists initially identified the leaking well in just one day using historical sensor patterns.
- Today, automated systems process this real-time data to detect or predict leaks in minutes, saving the company millions of dollars.

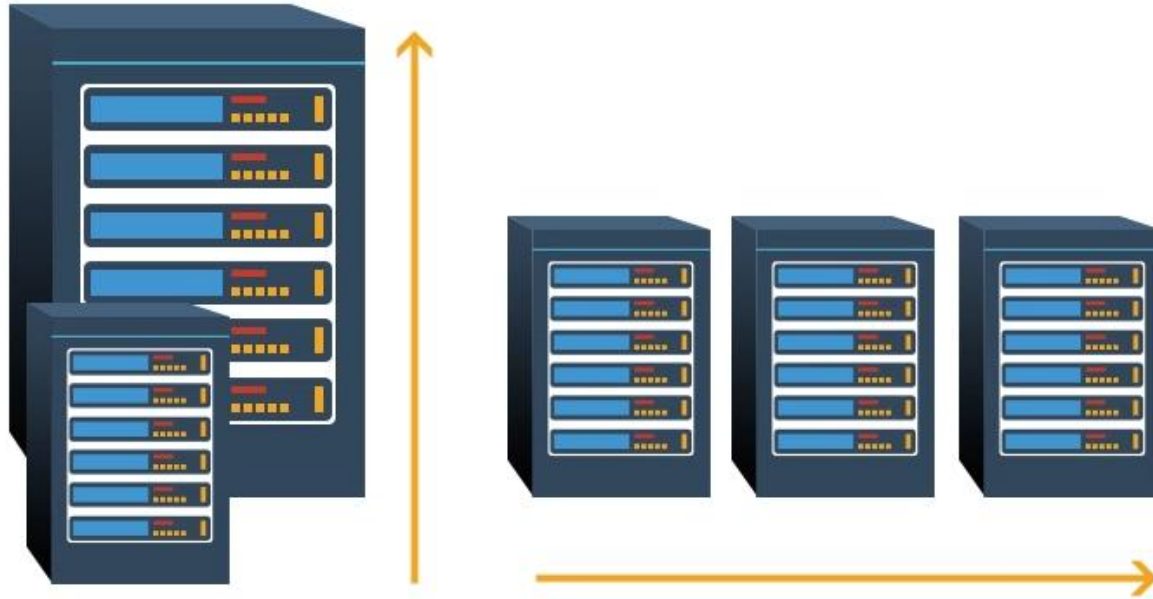


Use Case: Uber - Pay Surge Pricing if Battery is Low



Big Data Challenges

Big Data Challenges: Vertical Vs Horizontal Scaling

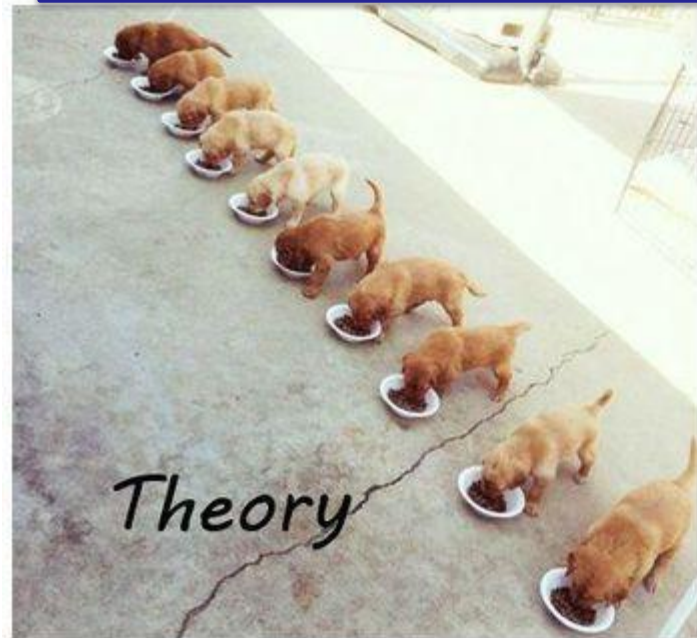


Vertical Scaling
(Scaling up)

Horizontal Scaling
(Scaling out)

Big Data Challenges

Scaling



Big Data Challenges: Scale of infrastructure



Image Source: <https://datacenter.legrand.com>

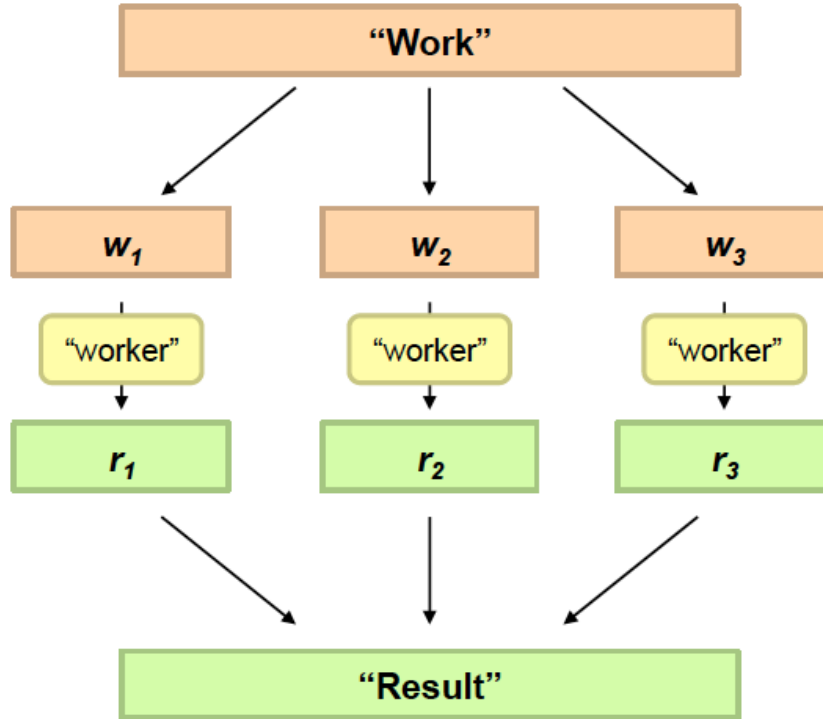
How to store *huge* files?

Requirements?

- Efficient Access
- Effective utilization of space
- Redundancy (Failsafe)
 - Given: probability of 1 disk failing is 1% per year
 - What are the chances that 1 out of 10^3 disk fails at a data center?

- Philosophy to Scale for Big Data?

Divide and Conquer



Divide Work



Combine Results

Distributed processing is non-trivial

- How to assign tasks to different workers in an efficient way?
- What happens if tasks fail?
- How do workers exchange results?
- How to synchronize distributed tasks allocated to different workers?



Big data storage is challenging

- Data Volumes are massive
- Reliability of Storing PBs of data is challenging
- All kinds of failures: Disk/Hardware/Network Failures
- Probability of failures simply increase with the number of machines ...

One popular solution: Hadoop



Hadoop Cluster at Yahoo! (Credit: Yahoo)

Hadoop

- Framework that allows for the distributed processing of large data sets
 - across clusters of computers
 - using simple programming models.
- Designed to scale up from single servers to thousands of machines, each offering local **computation** and **storage**.
- Designed to detect and handle failures at the application layer
 - delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures.

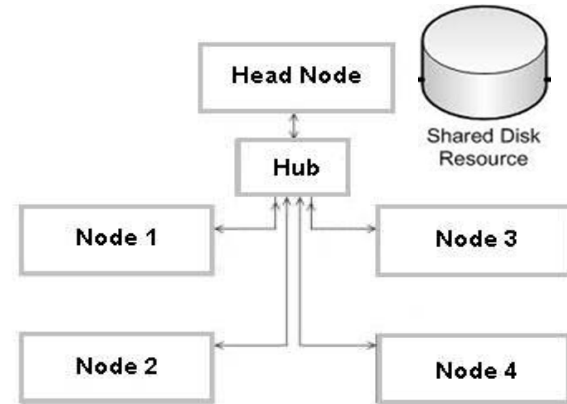
What is Hadoop?

- Hadoop is a framework that uses distributed storage and parallel processing to store and manage big data.
- It is the software mostly used by data analysts to handle big data
- There are three components of Hadoop:
 1. Hadoop HDFS - Hadoop Distributed File System (HDFS) is the storage unit.
 2. Hadoop MapReduce - Hadoop MapReduce is the processing unit.
 3. Hadoop YARN - Yet Another Resource Negotiator (YARN) is a resource management unit.

Hadoop Common - The common utilities that support the other Hadoop modules.

Motivation - Traditional Distributed systems

- ❑ Processor Bound
- ❑ Using multiple machines
- ❑ Developer is burdened with managing too many things
 - Synchronization
 - Failures
- ❑ Data moves from shared disk to compute node
- ❑ Cost of maintaining clusters
- ❑ Scalability as and when required not present



Hadoop: Myth Vs Truth

Myth	Truth
HDFS is a database	HDFS is a Distributed File System
Hadoop is a replacement of database warehouse	Compliments it, not a substitute
Hadoop is a complete, single product	Ecosystem, not just a product. HDFS and MapReduce being the key components
Hadoop is used only for unstructured data, web analytics	Enables many types of analytics

[illegible]

- **Meta (Facebook):** Manages massive user data, photo storage, and behavioral analytics.
- **Apple:** Powers Siri's voice recognition data processing and iCloud storage backends.
- **Amazon:** Drives product recommendation engines and e-commerce customer analytics.
- **Netflix:** Analyzes viewing habits to recommend content and optimize video streaming quality.

- **Visa:** Processes billions of credit card transactions to detect fraudulent activity instantly.
- **Mastercard:** Uses predictive analytics to secure payment networks and analyze merchant trends.
- **Bank of America:** Manages customer risk assessment, compliance reporting, and digital banking data.

- **Uber:** Calculates dynamic surge pricing, optimizes driver routes, and forecasts ride demand.
- **Airbnb:** Analyzes search metrics and user preferences to optimize property listing matches.



Nvidia

AI, GPU simulation, chip telemetry

Distributed processing ingests petabytes of GPU testing logs and simulation data.

Market cap > \$4.5T



Apple

Cloud ecosystems, internal ML

Thousands of nodes process search data, Siri NLP feedback, and iCloud storage optimization.

Market cap ~\$4T



Alphabet (Google)

Pioneer of MapReduce and GFS

Google Cloud Dataproc offers managed Hadoop for web indexing, YouTube analytics, and ad targeting.

Market cap ~\$3.7T



Microsoft

First-party managed cloud distro

Azure HDInsight runs Hadoop, Hive, Spark; internally handles telemetry from billions of installs.

Market cap ~\$3.5T



Amazon

Largest commercial Hadoop consumer

Amazon EMR drives e-commerce behavior analysis and product recommendation engines.

Market cap > \$2.2T

Hadoop offers

- Redundant, Fault-tolerant data storage
- Parallel computation framework
- Job coordination



Programmers

*No longer need to
worry about*



**Q: Where file is
located?**

**Q: How to handle
failures & data loss?**

**Q: How to divide
computation?**

**Q: How to program
for scaling?**

Evolution of HDFS

01

2006

HDFS and MapReduce were ceded to the Apache Software Foundation Project by its originators.

HDFS and Hadoop became available in version 1.0.

02

2012

03

2013

Version 2.0 of HDFS released; a general purpose YARN* resource manager was added; MapReduce and HDFS were decoupled.

Four alpha releases and one beta release of Hadoop.

2013-2017

04

05

2017

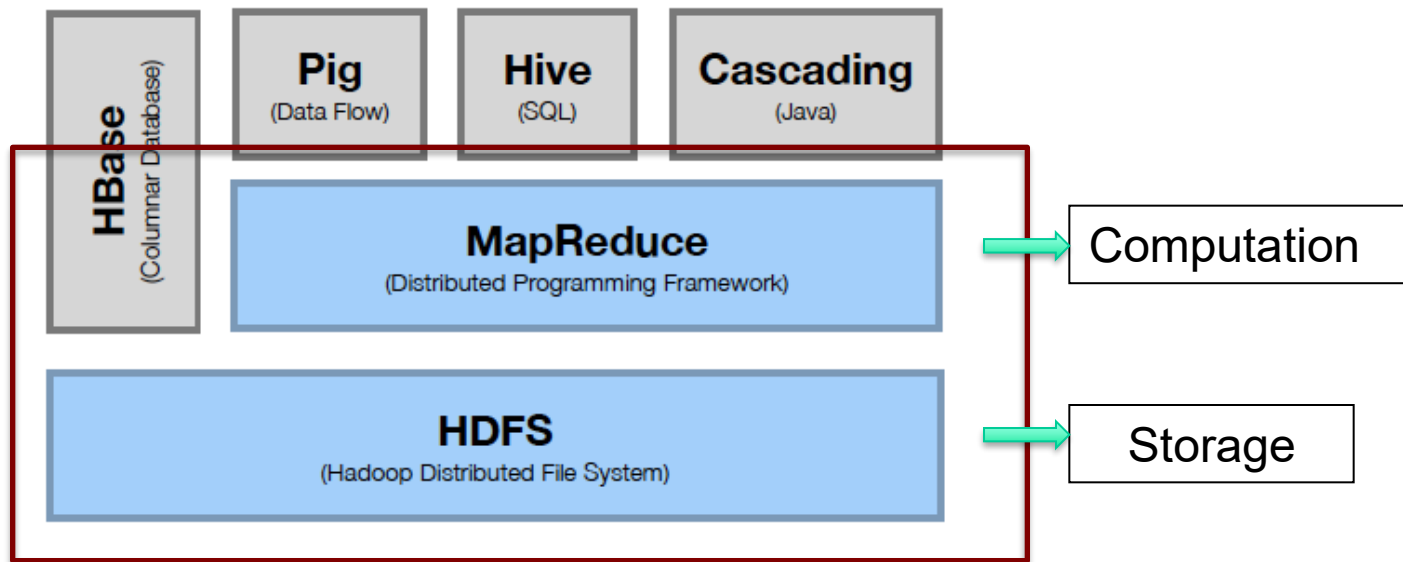
Apache Hadoop 3.0 became generally available with HDFS enhancements.

A little history on Hadoop

- Hadoop was started with **Doug Cutting and Mike Cafarella** in the year 2002 when they both started to work on Apache Nutch project.
- Apache Nutch project was the process of building a search engine system that can index 1 billion pages.
- The system will cost around half a million dollars in hardware, and along with a monthly running cost of \$30, 000 approximately.
- **In 2003**, they read white paper that described the architecture of Google's distributed file system, called **GFS (Google File System)** published by Google, for storing the large data sets.
- **In 2004**, Google published **MapReduce**.
- **In 2006**, Doug Cutting joined Yahoo along with Nutch project.
- **In 2007**, Yahoo successfully tested Hadoop on a 1000 node cluster and start using it.
- **In 2008, Yahoo released Hadoop as an open source project to ASF(Apache Software Foundation). And in July of 2008, Apache Software Foundation successfully tested a 4000 node cluster with Hadoop.**
- **In 2009**, Hadoop was successfully tested to sort a PB (PetaByte) of data in less than 17 hours for handling billions of searches and indexing millions of web pages.



Hadoop Stack



Motivation Questions

- **Problem 1:** Data is too big to store on one machine.
- **HDFS:** Store the data on multiple machines!

Motivation Questions

- **Problem 2:** Very high end machines are too expensive
- **HDFS:** Run on commodity hardware!

Motivation Questions

- **Problem 3:** Commodity hardware will fail!
- **HDFS:** Software is intelligent enough to handle hardware failure!

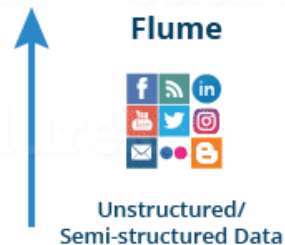
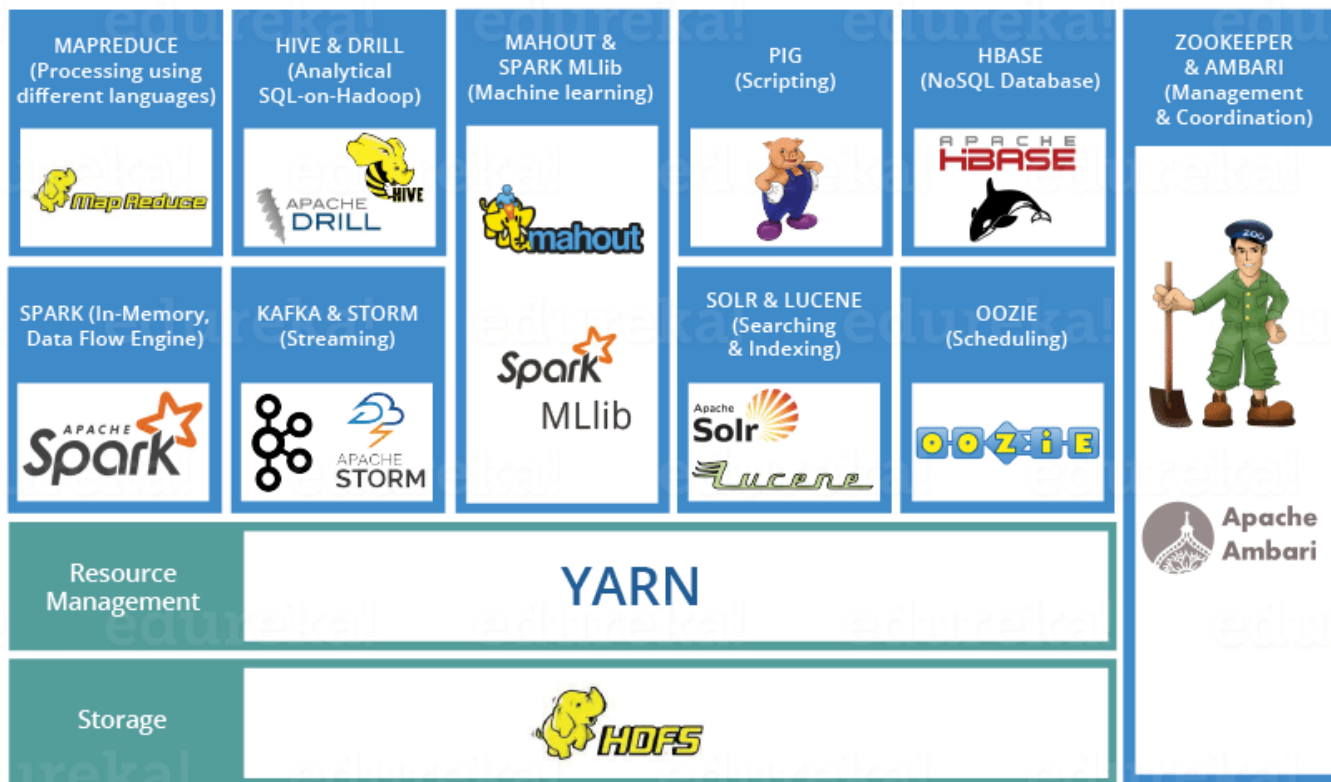
Motivation Questions

- **Problem 4:** What happens to the data if the machine stores the data fails?
- **HDFS:** Replicate the data!

Motivation Questions

- **Problem 5:** How can distributed machines organize the data in a coordinated way?
- **HDFS: Master-Slave Architecture!**

Hadoop Ecosystem



- There are *four major elements of Hadoop* i.e. **HDFS, MapReduce, YARN, and Hadoop Common**.
- Following are the components that collectively form a Hadoop ecosystem:
 - HDFS: Hadoop Distributed File System
 - YARN: Yet Another Resource Negotiator
 - MapReduce: Programming based Data Processing
 - PIG, HIVE: Query based processing of data services
 - HBase: NoSQL Database
 - Mahout, Spark MLlib: Machine Learning algorithm libraries
 - Solar, Lucene: Searching and Indexing
 - Zookeeper: Managing cluster
 - Oozie: Job Scheduling

HDFS

Hadoop distributed File System

HDFS

- Data storage system used by Hadoop
 - Hadoop: Project to develop open-source software for reliable, scalable, distributed computing★
- Components
- Architecture
- Tasks / Services

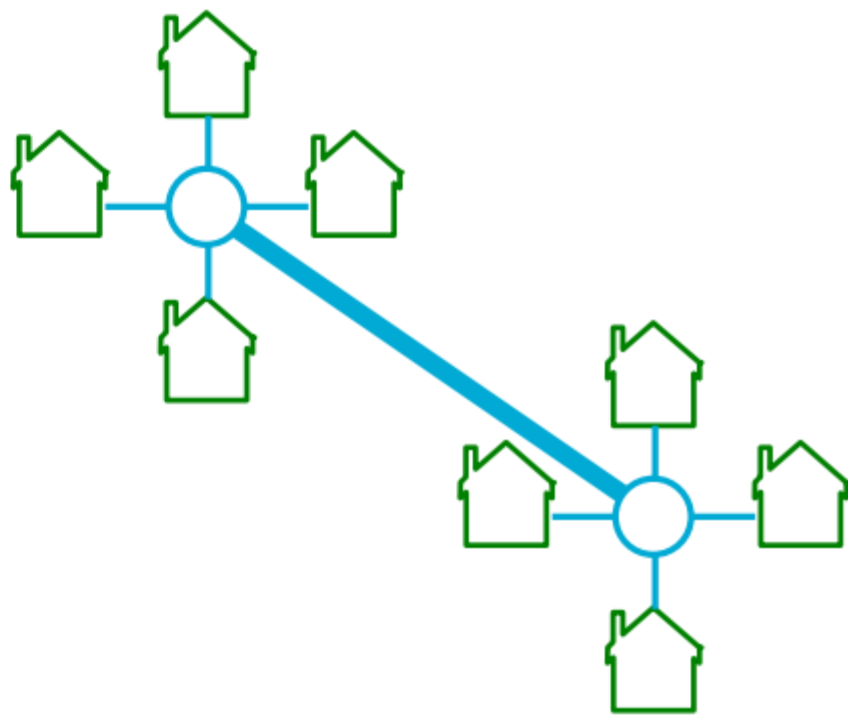
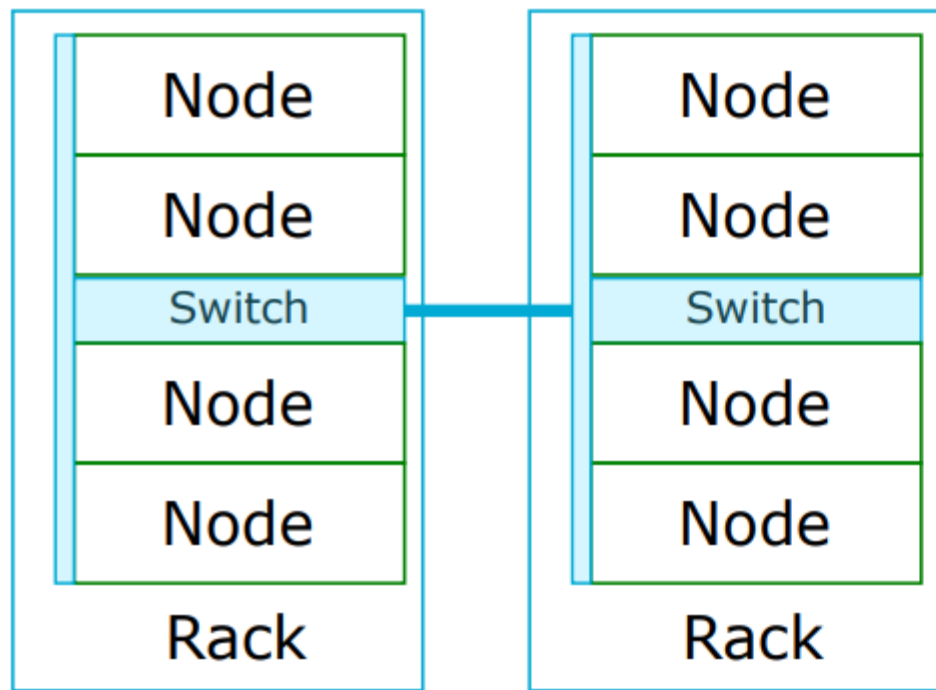
Hadoop

Hadoop framework allow us to store large volume of data at multiple location

Hadoop framework also allow us to process data in parallel manner

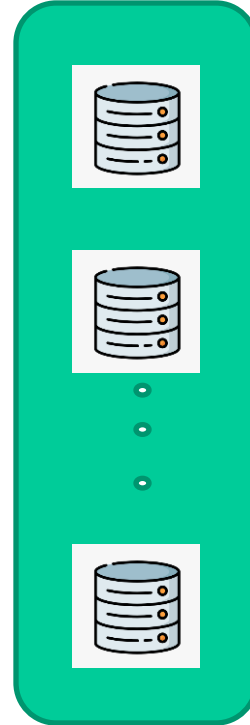
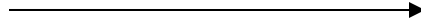
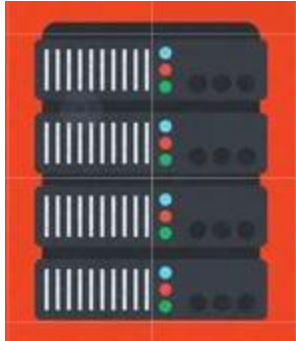
HDFS Physical Infrastructure

- ❑ Racks
- ❑ Switch
- ❑ Cluster



HDFS

Hadoop Distributed File System (HDFS) is a specially designed file system for storing huge dataset in commodity hardware.



- ❑ Hadoop Distributed File System (HDFS) is the open source version of the Google File System (GFS)
- ❑ Allows reliable and efficient storage and access of large write-once, read-many (WORM) files
- ❑ NameNode acts as a server that manages the filesystem
- ❑ DataNodes store data blocks and serve read/write requests
- ❑ Blocks are replicated to allow for fault tolerance and fast reads
- ❑ HDFS allows data to be written only once to a file. Files cannot be modified after creation, although appending data may be supported if enabled.

HDFS has two main components

Data node

Name node

Components of HDFS



Secondary NameNode



Active NameNode



Standby NameNode



DataNodes

Name Node

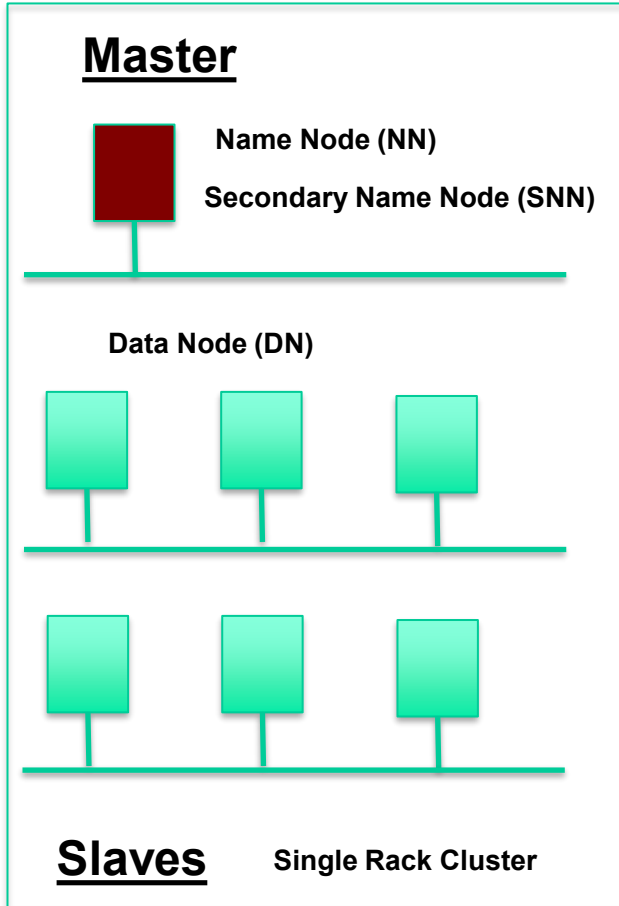
- ❑ The NameNode is a critical component of HDFS and serves as the master server.
- ❑ It is responsible for managing the metadata of the file system.
- ❑ This metadata includes information about the structure of the file system, the location of data blocks, and the health of DataNodes.

Name Node

- ❑ The NameNode does not store the actual data content of files but keeps track of the metadata, such as file names, permissions, and the hierarchy of directories.
- ❑ It maintains an image of the entire file system namespace and the mapping of data blocks to DataNodes.
- ❑ The **NameNode** manages the file system namespace by performing operations such as opening, closing, and renaming files and directories.

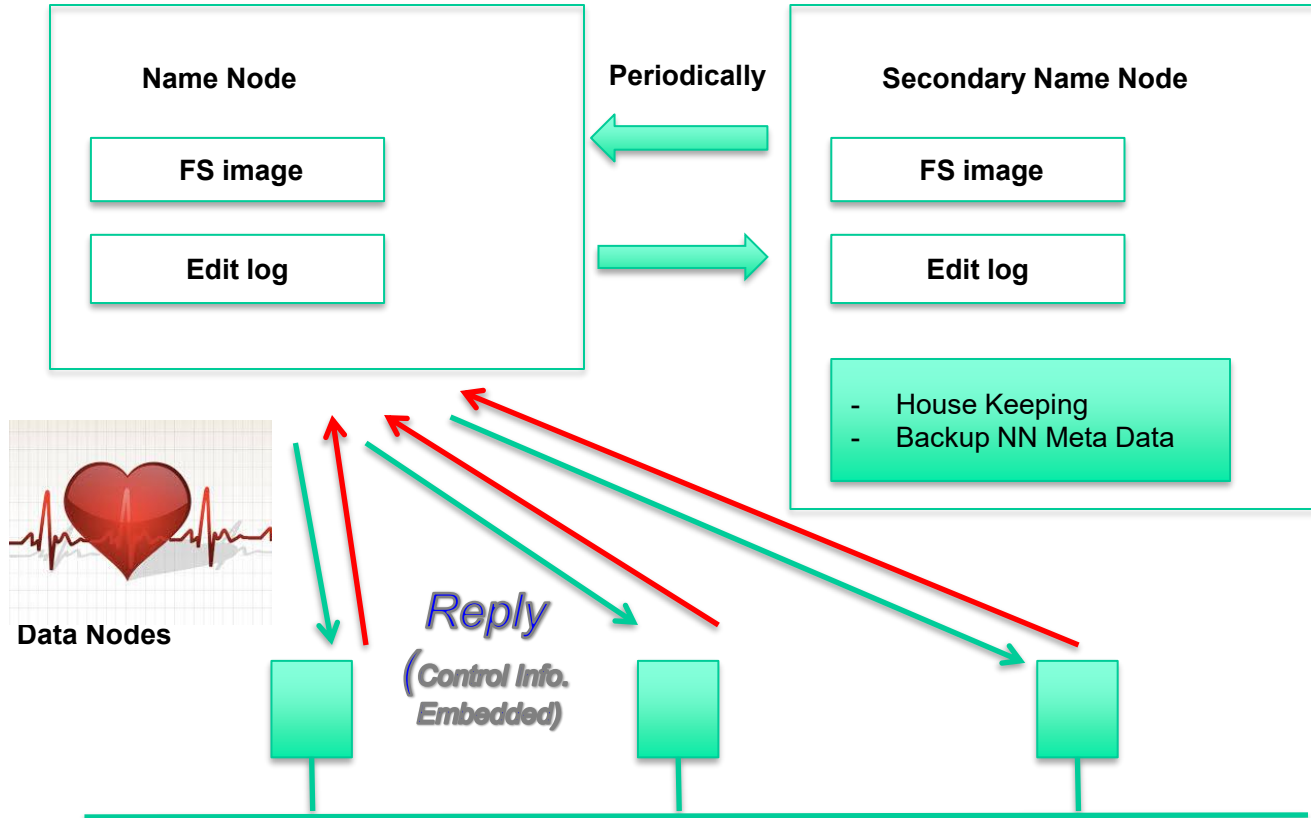
- ❑ Editlog : keep track of recent changes
- ❑ Fsimage : Keep track of every changes made on HDFS since the beginning

HDFS Architecture: Master-Slave



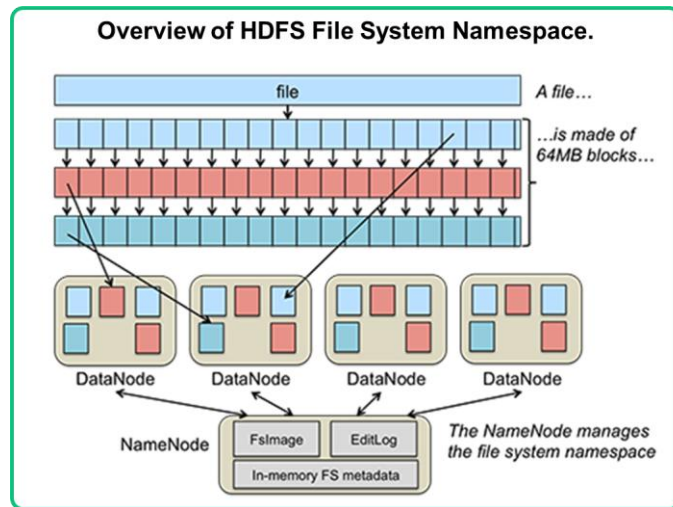
- Name Node: Controller
 - File System Name Space Management
 - Block Mappings
- Data Node: Work Horses
 - Block Operations
 - Replication
- Secondary Name Node:
 - Checkpoint node

HDFS Inside: Name Node



Heart beating every 3 seconds. If NN does not hear from DN in 10 mins, it starts to replicate the blocks

HDFS Inside: Blocks



Q: Why do we need the abstraction “Blocks” in addition to “Files”?

Reasons:

- File can be larger than a single disk
- Block is of fixed size, easy to manage and manipulate
- Easy to replicate and do more fine grained load balancing

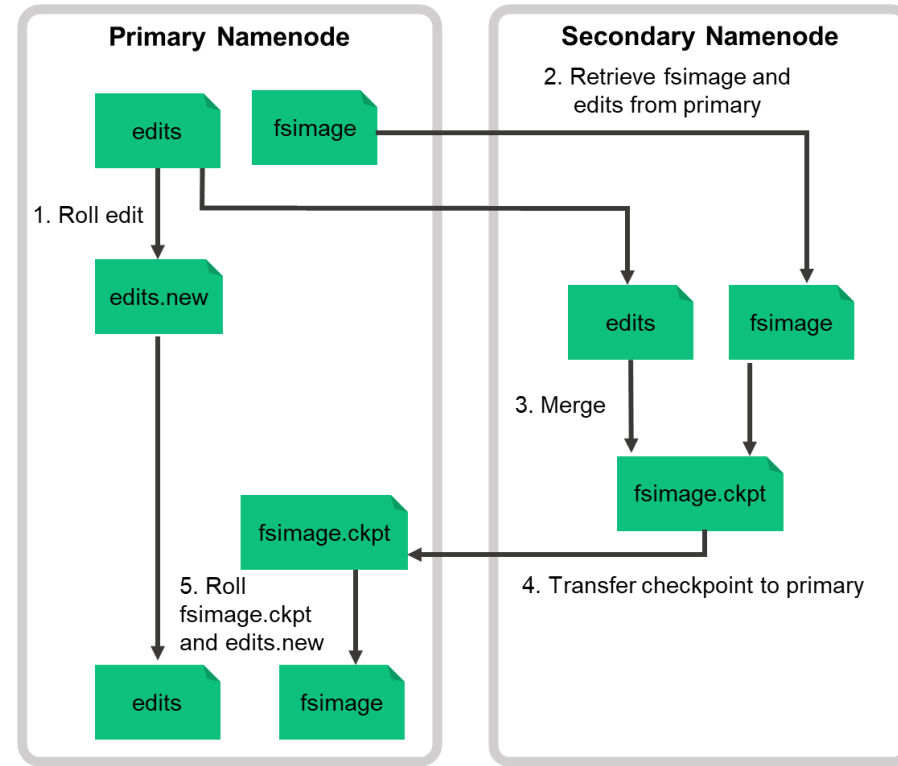
Note: The content of the file is usually split into large blocks of 128 MB size, but user has the flexibility of selecting the size for each file.

Secondary NameNode

- The **Secondary NameNode** is **not a backup or standby server** for the Primary NameNode (it cannot take over if the Primary NameNode crashes).
- Instead, its primary function is to act as a **checkpoint helper** to prevent the Primary NameNode's log files from growing too large and to speed up cluster startup times.

Secondary Namenode

- The important job of secondary NameNode is checkpointing.
- Secondary NameNode merges the copy of *fsimage* and *edits* the file for primary Namenode. The secondary namenode downloads the edits file from primary and applies/merges it with '*fsimage*' and then sends it back to the primary NameNode.
- The secondary NameNode saves the latest checkpoint in a directory which is structured the same way as the primary NameNode's directory. This is to ensure that the checkpointed image is always available to be read by the primary NameNode if necessary.



Editlog : keep track of recent changes made to the meta information. In new versions of Hadoop, there would be multiple **edit** files (per transaction) that get created which store the changes made to meta.

Fsimage : Keep track of every changes made on HDFS since the beginning. This is master copy of the metadata for the file system.

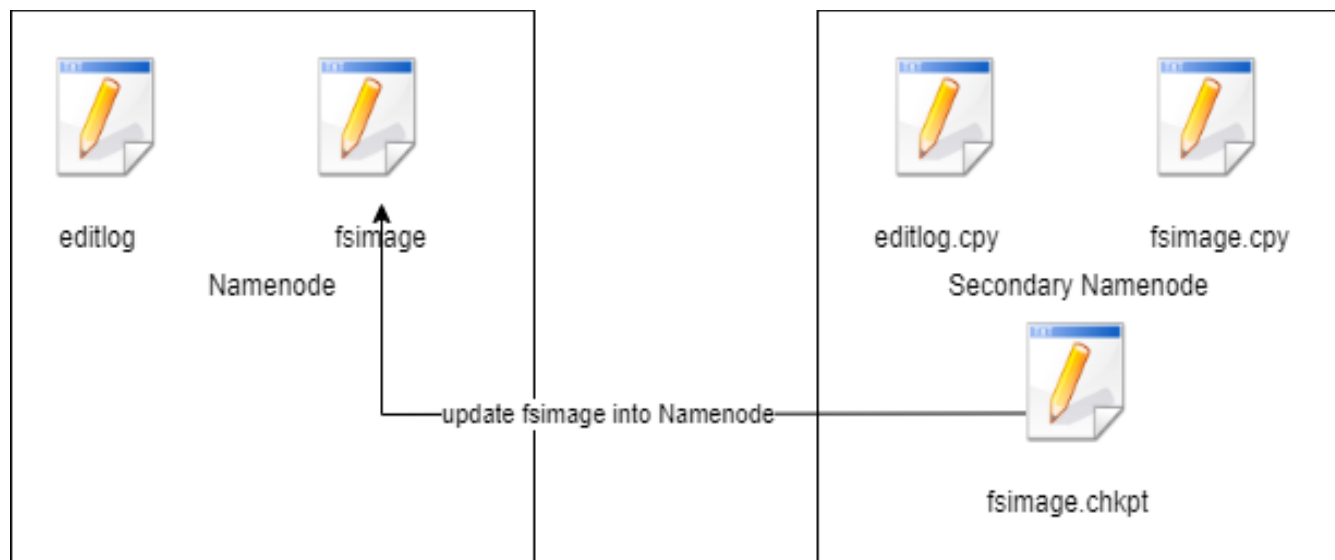
Core Responsibility of Secondary Name Node: Merging Metadata (Checkpointing)

- The Primary NameNode keeps the active file system metadata in RAM for fast access.
- It logs changes in real-time to an **EditLog** file on disk and periodically saves a snapshot of the file system structure to an **FsImage** file.

The Secondary NameNode performs the heavy lifting of merging these two files:

- **Fetch:** Periodically downloads the current FsImage and EditLog from the Primary NameNode over the network.
- **Merge:** Combines the FsImage and EditLog in its own memory to create a brand-new, updated FsImage (called a checkpoint).
- **Upload:** Sends the new FsImage back to the Primary NameNode.
- **Reset:** The Primary NameNode replaces its old FsImage with the new one and truncates the EditLog.

Secondary Namenode



Secondary NameNode vs. Standby Name Node

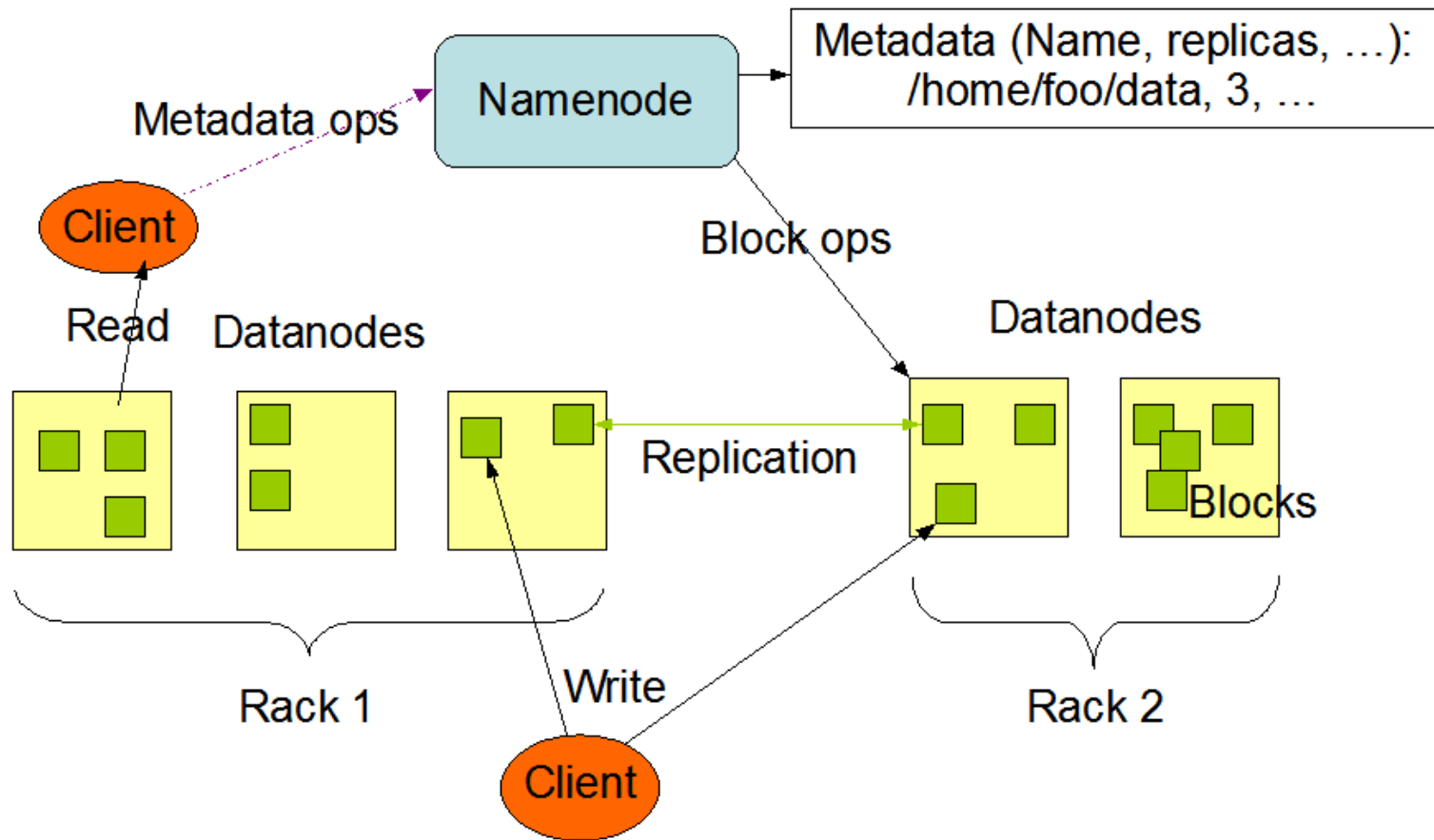
Feature	Secondary NameNode (Classic Architecture)	Standby NameNode (Hadoop 2.x+ HA Setup)
Role	Helper process for periodic checkpointing	Active hot-standby replica
Automatic Failover?	No. Cannot take over if Primary fails.	Yes. Automatically takes over instantly.
Data Freshness	Outdated (only up to the last checkpoint).	Real-time synchronized.

Data Node

- ❑ DataNodes are worker nodes that store and manage data blocks in the Hadoop Distributed File System (HDFS).
- ❑ DataNodes are primarily responsible for storing data, serving client read/write requests, and sending heartbeats and block reports to the NameNode.
- ❑ A Hadoop cluster can have one, hundreds, or even thousands of DataNodes, depending on its size and requirements.
- ❑ As the number of DataNodes increases, the storage capacity and data processing capability of the Hadoop cluster generally increase.

- ❑ A file is split into one or more blocks(e.g. 128 MB), and these blocks are stored across multiple DataNodes.
- ❑ DataNodes are responsible for serving read and write requests from HDFS clients.
- ❑ DataNodes also perform block creation, deletion, and replication as instructed by the NameNode.

HDFS Architecture



DataNodes

❑ Block report

A Block Report in HDFS is a report sent by each DataNode to the NameNode that contains information about all the data blocks stored on that DataNode

❑ Heartbeats

A periodic signal sent by a DataNode to the NameNode to indicate that it is alive and healthy. If heartbeats stop for a specified time, the NameNode marks the DataNode as failed and initiates block replication to maintain data availability.

Heartbeats

Purpose: Confirms that the DataNode is active and healthy.

Sent by: DataNode

Received by: NameNode

Frequency: By default, every **3 seconds** (configurable).

Contents: A heartbeat includes the DataNode's status, such as:

- Available storage space
- Ongoing data transfers
- Health status

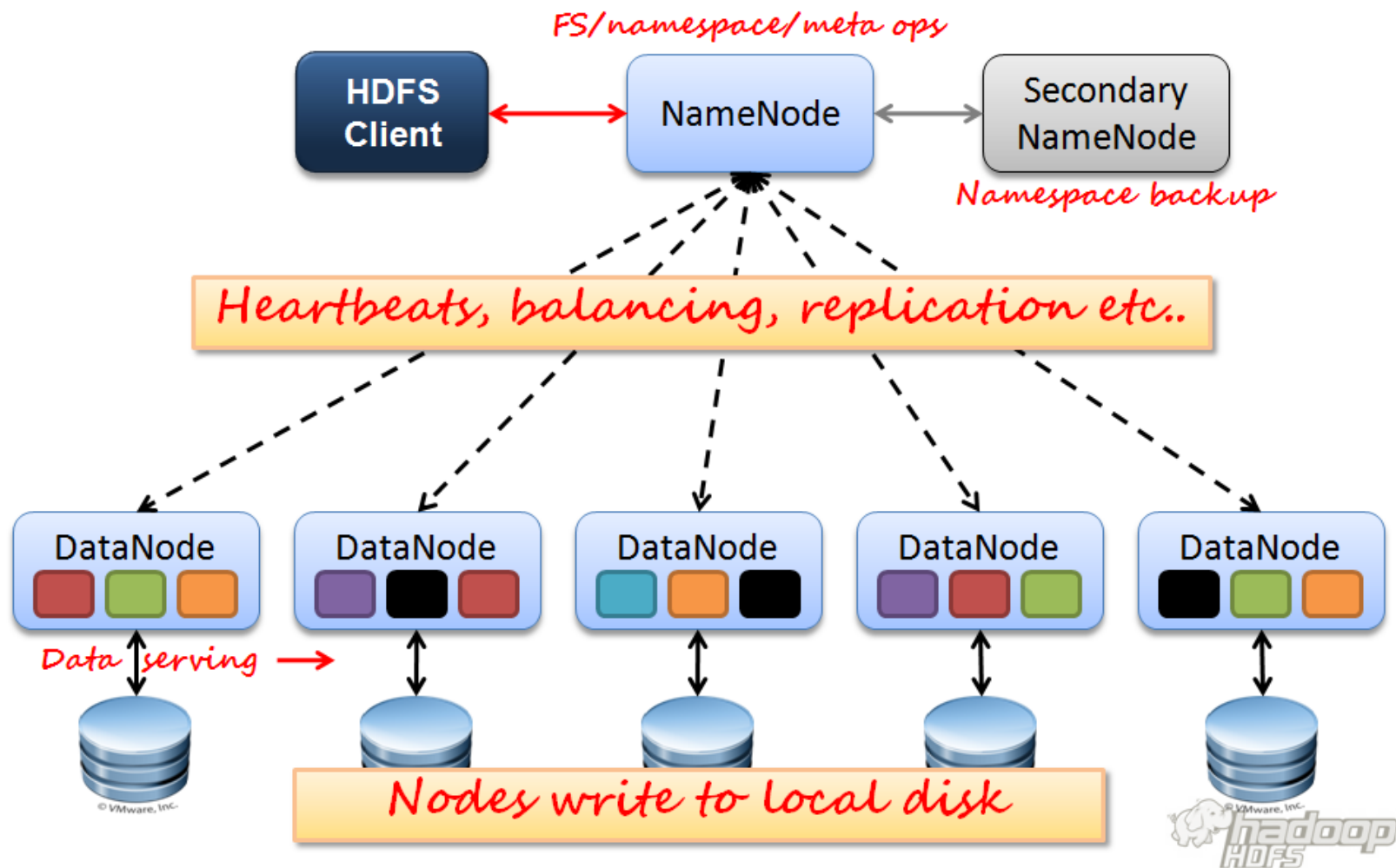
What happens if a heartbeat is missed?

- ❑ If the NameNode does not receive heartbeats from a DataNode for a specified period (by default, about **10.5 minutes**), it marks the DataNode as **dead**.
- ❑ The NameNode then replicates the missing data blocks to other healthy DataNodes to maintain the configured replication factor.

Heartbeat vs. Block Report

Heartbeat	Block Report
Sent every ~3 seconds	Sent periodically (by default, about every 6 hours)
Confirms the DataNode is alive	Lists all blocks stored on the DataNode
Used to monitor node health	Used to verify block locations and metadata

Architecture



Storing file on HDFS

- Default replication factor is 3
 - first replica of a block will be stored on a local rack
 - the next replica will be stored on a remote rack
 - the third replica will be stored on the same remote rack but on a different Datanode
- More replicas?
 - the rest will be placed on random Datanodes
 - As far as possible, no more than two replicas are kept on the same rack

Tasks of NameNode

❑ Manages File System

- mapping files to blocks and blocks to data nodes

❑ Maintaining status of data nodes

➤ Heartbeat

- Datanode sends heartbeat at regular intervals
- If heartbeat is not received, datanode is declared dead

➤ Blockreport

- DataNode sends list of blocks on it
- Used to check health of HDFS

The Namenode does not directly call Data Nodes. It uses replies to heartbeats to send instructions to the Data Nodes.

The instructions include commands to:

- ❑ replicate blocks to other nodes;
- ❑ remove local block replicas;
- ❑ re-register or to shut down the node;
- ❑ send an immediate block report

NameNode Functions

- ❑ Replication
 - On Datanode failure
 - On Disk failure
 - On Block corruption
- ❑ Data integrity
 - Checksum for each block
 - Stored in hidden file
- ❑ Rebalancing - balancer tool
 - Addition of new nodes
 - Decommissioning
 - Deletion of some files

- ❑ The primary objective of HDFS is to store data reliably in the presence of failures
- ❑ Three common failures that it must handle are:
 - DataNode failure
 - NameNode failure
 - Network partition

Data integrity

What if a block of data fetched from a DataNode arrives corrupted

Fault in a storage device

- Network faults
- Buggy software

An HDFS client creates a checksum for every block of its file and stores it in the HDFS namespace

- When the client retrieves the contents of a file it verifies that they match ,if not, it must retrieve the block from another replica

Cluster Rebalancing

- ❑ HDFS architecture is compatible with data rebalancing schemes
- ❑ A scheme may automatically move data from one DataNode to another if the free space on a DataNode is falling below a certain threshold

Replication-Pipeline

- ❑ When a client is writing data to an HDFS file with a replication factor of three, the NameNode retrieves a list of DataNodes using a replication target choosing algorithm.
- ❑ It contains the DataNodes that will host a replica of that block.
- ❑ The client streams data in portions (packets) directly to the first DataNode, which then immediately forwards each portion to the second DataNode in the pipeline.
- ❑ The second DataNode, in turn starts receiving each portion of the data block, writes that portion to its repository and then flushes that portion to the third DataNode.
- ❑ Finally, the third DataNode writes the data to its local repository.
- ❑ Thus, the data is pipelined from one DataNode to the next.

Replication Pipeline Steps

Step 1: Client Initiates Write

- The client contacts the NameNode to request file creation.
- The NameNode checks permissions and provides a list of DataNodes where the block should be stored.

Step 2: Block Replication Pipeline Formation

- The client writes data to the first DataNode (DN1).
- DN1 forwards the data to the second DataNode (DN2).
- DN2 forwards the data to the third DataNode (DN3).
- This forms a pipeline (Client → DN1 → DN2 → DN3).

Step 3: Acknowledgment (ACK) Sent Backwards

- Once DN3 receives the data, it sends an ACK (acknowledgment) back to DN2.
- DN2 forwards the ACK to DN1.
- DN1 sends the final ACK to the client.

Step 4: Success Confirmation

- The client receives the ACK, confirming successful replication.
- If any DataNode fails, the pipeline adjusts dynamically.

Advantages of Pipelined Replication

- Faster writes (parallel streaming instead of sequential).
- Reduced network congestion (data flows in one direction).
- Automatic recovery if a DataNode fails during replication.



Understanding HDFS Safemode

What is Safemode?

- Safemode is a **read-only administrative state** entered automatically by the NameNode during startup.
- In this mode, no file system modifications, block replications, or block deletions can occur.

Why is it Required?

- Because the NameNode does **not persist block locations** on disk.
- During startup, it relies on incoming Block Reports from all DataNodes to rebuild its in-memory block mapping safely before serving write traffic.

Safemode Operational Workflow

- **Read-Only State:** At startup, no block replication, modification, or deletion is allowed.
- **Block Reports:** NameNode receives heartbeats and full block inventories from registering DataNodes.
- **99.9% Threshold Check:** NameNode verifies that of total blocks are reported.
- **Post-Safemode Repair:** Missing or under-replicated blocks are replicated *only after* Safemode exits.

HDFS Boot & Safemode Exit Sequence

1. NameNode Boot

Loads fsimage and editlog into memory. Enters Safemode automatically.

READ-ONLY

2. Block Registration

DataNodes establish connections and send full block reports containing local replica locations.

99.9% TARGET

3. Exit & Active Writes

After threshold satisfaction and a 30-second extension timer (), full cluster write operations resume.

ALL IS WELL

HDFS Robustness

- ❑ Safemode
 - At startup: No replication possible
 - Receives Heartbeats and Block reports from Datanodes
 - Around 99.9% of blocks are checked for defined replication factor
- ❑ Replicate blocks wherever necessary

All is well 😊 → Exit Safemode

Moving Data in and out: Useful Commands

Moving Data into HDFS

- ❑ **hadoop fs -put / hdfs dfs -put** : Copies files from the local filesystem to HDFS.
 - `hadoop fs -put local_file /hdfs/path/`
- ❑ **hadoop fs -copyFromLocal**: Similar to -put, but restricts the source to the local filesystem.
 - `hadoop fs -copyFromLocal local_file /hdfs/path/`
- ❑ **hadoop fs -moveFromLocal**: Moves a file from local storage to HDFS (deletes local file after copying).
 - `hadoop fs -moveFromLocal local_file /hdfs/path/`
- ❑ **hadoop fs -appendToFile**: Appends local file content to an existing HDFS file.
 - `hadoop fs -appendToFile local_file /hdfs/existing_file`

Moving Data out of HDFS

- ❑ **hadoop fs -get / hdfs dfs -get**: Copies files from HDFS to the local filesystem.
 - `hadoop fs -get /hdfs/file local_destination`
- ❑ **hadoop fs -copyToLocal**: Similar to -get, but restricts the destination to the local filesystem.
 - `hadoop fs -copyToLocal /hdfs/file local_destination`
- ❑ **hadoop fs -getmerge**: Merges multiple HDFS files into a single local file.
 - `hadoop fs -getmerge /hdfs/folder/* merged_local_file`

HDFS Summary

- ❑ Fault tolerant
- ❑ Scalable
- ❑ Reliable
- ❑ File are distributed in large blocks for
 - Efficient reads
 - Parallel access

Further Reading

- [A Brief History of Big Data Everyone Should Read](#)
- [Beyond Volume, Variety and Velocity is the Issue of Big Data Veracity](#)
- What is big data? - [OpenSource.com](#) & [O'Reilly](#)
- [Uber Use Case](#)
- [5 Big Data Use Cases To Watch](#)
- [Best Big Data Analytics Use Cases](#)
- [The 5 game changing big data use cases](#)
- [Big Data - The 5 Vs Everyone Must Know](#)
- [Top SlideShare Presentations on Big Data](#)
- [Google Data Center 360° Tour](#)